

Giovanni's blog

Monday, June 8, 2026

I'm building a parallel internet, and it's called The Thinnernet



Since 2020, I've taken on a Steve Jobs alter-ego (on a *very, very* part time basis, and often for [humor purposes](#)). The first essay I wrote wasn't even related to hardware, or solar powered-conscious operating systems that began around the same time. It was actually about one of my former jobs, regarding Knowledge Bases. Using one every day, I thought there *had* to be a better way to integrate steps taken into a ticket management system, to show proof of work. I won't say which system and which company, since it doesn't matter. I thought back at the time, where else did someone care so much about not only user experience, but the employee's experience? And I realize it was Steve Jobs. [Here is the essay](#) I wrote, on a higher level Knowledge Base called Experience Base, later included in part of a concept CMS called TicketMS.

A month ago, I read an article by Dr. Nathalie Martinek that basically described the exact problem my organization had- inability or unwillingness, or slowness to adopt new or improved changes to a workflow.

If you were a futurist and tried to predict the future using only today's news headlines as a indicator, you might think that the future will consist of a bunch of buzzing drones delivering Doritos and Dr. Pepper. That is for the most part true. However, what isn't really flashy or covered is what the internet will/might look like. Fiber optic tech has seen significant improvements, and it's likely that more households will have access to something like 1Tbps before 2050. But very few people will need that much data- maybe- I suppose 125GB/s might be useful for 16K uncompressed video, but I am not sure what else. Maybe lots of coordination feeds with other autonomous agents.

In 2023, I started thinking and writing about a parallel internet:

Blog Archive

[June 2026](#) (5)

[May 2026](#) (1)

[April 2026](#) (4)

[January 2026](#) (2)

[December 2025](#) (3)

[September 2025](#) (1)

[August 2025](#) (1)

Report Abuse

Search This Blog

[Home](#)

I'm building a parallel internet, and it's called The Thinnernet



Popular Posts

I'm building a parallel internet, and it's called The Thinnernet

Since 2020, I've taken on a Steve Jobs alter-ego (on a very, very part time basis, and often for humor purposes). The ...

The State of Stateless Linux

History and definition An Audacious Plan to Halt the Internet's Enshittification by Cory Doctorow at DEF CON 31, 2023
The term enshittification was coined by Doctorow in a November 2022 blog post^[1] that was republished in Locus in January 2023.^[2] He expanded on the concept in another blog post,^[3] which was republished in the January 2023 edition of Wired.^[4]

In a 2024 article on ft.com, Doctorow extended the word with the term "enshittocene" to state that "enshittification" is coming for absolutely everything".^[5]

The beginning of an enshittification & AGI/ASI era necessitates a parallel internet infrastructure endeavor. Enter Minitel 2W. Local-first, internet where text and encyclopedias are prioritized over Advertisements, Algorithms, and antagonistic partisanship. One where feces is absent. Publically funded internet in the public interest.

OSes, and the future of Solar Computing

The title is more of a gag on the other parodied State of the Union addresses, like State of the Onion by Perl Developer Larry Wall: <https://www.perl.com/news/feature/state-of-the-onion>

How about new Java based phone apps?

(and a new Symbian-like RTOS while you're at it)

You might also be wondering, wasn't Steve Jobs more of a device guy, and not an infrastructure or web design auteur? And the answer is yes! While he was one of the first to include a TCP/IP stack in his Unix OS & hardware in 1987, he wasn't really focused on laying down internet undersea cables. There was plenty of optimizing the user experience on the personal computer alone. Most desktop programs didn't even need an internet connection, and some still don't. Plus, internet traffic at the time was very limited, and there was very little damage any "CDN" or website could do. I say that because there was so little data being transferred, that few limitations could be placed on files, formats, protocols, browsers, and services.

If you were to wonder what a 21st century (post 2011) Steve Jobs "type" wanted to accomplish, he would be focused on the same exact thing: The user experience. But the user experience today is being shaped less by hardware and more the infrastructure and the social networks that many are dwelling on. Often times, still voluntarily. If Steve Jobs really wanted to ensure that users could get the same user experience as they reliably could on a spotty, but flat delivery of data on a 2G or 3G connection, he would be focused much more on ensuring the apps *could* use limited data. Because internet speed is part of the user experience.

There is a funny email that had been released ~~after~~ before Jobs's passing where a user complained of a spotty signal, and his advice was basically to not hold the phone in that direction (or with his hand over the top part where the antenna was positioned). One of the most influential CEOs at the time was basically suggesting rabbit ears fiddling like on analog television receivers. But I defend his actions, because 3G technology at the time wasn't robust, and one shouldn't have expected him to have all the solutions that were out of his control (even if they tried to build their own modem and it was inadequately developed compared to Qualcomm's, which DID happen, but that's not really a major part of the current issue). It's that all technologies take time to mature, and once a lot of the EMI kinks are worked out, a certain level of reliability becomes expected. And I have absolutely no proof, but I think that would be one new frontier that, had Jobs lived to 70 or 80, might have embarked on.

What am I referring to? In order to ensure the user experience is consistent across every device, the entire transmission of application data/content would have to be completed in a real time deadline. Asynchronous communication is certainly welcome and important, but many features can be standardized to a point where digital interactions become constructable and replicable. Refresh rate is something iPhones have mastered with latency. But internet latency is a different story. It can be important when accessing a website or service and hoping to get the same format view, with the same data loading in the same amount of time. With static HTML, the page might be wide and the text small, because the browser has no auto-resize feature. But at least it might not have any problems loading the text (unlike a browser that can't display Javascript), and the text can be converted into a different view, like Reading View, which modern browsers offer.

I wouldn't say this is leading to a homogenized user experience. What I think is lost in the era where everything requires a ton of bandwidth is that it is very hard to notice how much data is being transferred unless you're on a slow connection. Admittedly, I am testing a slow connection out, not that I am complaining about the connection speed when I have the option to upgrade. But that is not why I want others to try out a Thinnernet. It's because I'm suggesting a fallback in case the infrastructure can't handle an always on, always connected world where hundreds of GB are downloaded a day, yet can't deliver a single byte in an instance where there is an outage or an easily preventable network congestion issue.

So what is Thinnernet? Imagine a fiber optic bundle of undersea cables- maybe a hundred or so 10Gbps cables comprising a large, round internet cable with a diameter of maybe 3 feet (highly weatherproof). These might have been manufactured 15 years ago. Today, they're likely higher capacity. Now imagine you have a very advanced computer from 2040 where your PC's ethernet port can receive that speed all at once with a single cable- maybe 1Tbps or 10Tbps. Now return to

the original bundle, and pick one cable. That's your internet connection. It might be 10Gbps, even though the ISP can sell you a service for 10Tbps.

Now instead of that 10Gbps cable, go back to the mid-1990s, and an undersea coaxial cable company is using maybe 10Mbps in each bundle for a total of 100Mbps or 1Gbps. Now connect a single coax cable to your PC (using the right modem or ethernet converter). You have 10Mbps.

If you're Steve Jobs, and you want every application to have the same predictable experience, whether someone is using 1 Mbps, 10Mbps, or 1Gbps, there would need to be a minimized mode of traffic that ensures a certain set of apps can predictably access a subset of websites and "essential data" with known webpage sizes (e.g. under 100KB, or under 10KB), so they are more or less whitelisted for usability, and no guesswork by the user has to be done to figure out what sites can be accessed on a certain connection. The way the web is designed is still open, and rightfully so, with decentralized servers and DNSes. But applications themselves and the servers they access must be able to know where they can be used reliably. This is not really a single website's or network engineer's problem, but everyone's. A big fish in a small pond is going to use a lot more data in a place where little data is typically used. In the mid 2010s, you could still find a lot of people using Symbian phones. At one point, I set up a twitter relay service on my phone, which allowed me to read and send tweets using just SMS and without an internet data plan! I recall Facebook offered this in India for relaying to their Messenger services but eventually got rid of it due to the costs.

In theory, using less data shouldn't be much of an issue or cost for a big company. But because it uses legacy infrastructure, or has fewer spam mitigation tools (e.g. with Carriers slow to ban numbers rather than having a software dashboard by a Meta employee) the costs are much higher than when using a higher speed technology such as Fiber. The same reason Verizon and other phone companies gradually phased out copper phone lines.

History may be often boring, but it also offers alternatives and ideas that have long been forgotten. The internet as it was built today includes some very fast and amazing technology, but it also includes a lot of deliberately crummy software designed to get people to upgrade. While some people think this a healthy feature of capitalism doing its job, its also limiting the idea of capitalism to incumbent builders who are prioritizing bandwidth heavy platforms and not really offering an energy-lite way to access the web. An adaptable internet is really the best user experience a company can offer. Which is why I think Steve Jobs would have been the Cornelius Vanderbilt of the Information Superhighway, had he not died of cancer. And by that, I mean that everyone who could afford to ride on one of this trains or railroads would all get the same speed. Thomas Friedman once wrote

“Communism was a great system for making people equally poor. In fact, there was no better system for that than communism. Capitalism made people unequally rich.”

When people try to describe low tech or high tech, they inevitably use an ahistorical starting point to describe slow internet or a slow computer using terms or concepts that are often non-technical. The people who know how to run a fast internet on a slow data connection like me would be considered minimally proficient. The people who cannot get ANYthing to run on a slow data connection are non-proficient. The difference may be subtle, but new technology adoption often depends on a "lie" that something is broken or antiquated. While there are many new technology advances that might not have been possible had too few people adopted a highly profitable new product, it doesn't erase the historical truths of the efficiency and even security (not always) of older systems.

And for that reason, a competent engineer could design five different speeds where the most important data still arrives first, allowing the least bandwidth tiered user to still receive a static HTML of their email, while the highest paying customer receives a tacky, luxurious Javascript wrapper around their inbox (arriving at the same time, like that feather and hammer drop experiment on the moon):

"Design reform began with Exhibition organisers [Henry Cole](#) (1808–1882), [Owen Jones](#) (1809–1874), [Matthew Digby Wyatt](#) (1820–1877), and [Richard Redgrave](#) (1804–1888),^[11] all of whom deprecated excessive ornament and impractical or badly-made things.^[12] The organisers were "unanimous in their condemnation of the exhibits."^[13] Owen Jones, for example, complained that "the architect, the upholsterer, the paper-stainer, the weaver, the calico-printer, and the potter" produced "novelty without beauty, or beauty without intelligence."^[13] From these criticisms of manufactured goods emerged several publications that set out what the writers considered to be the correct principles of design. Richard Redgrave's Supplementary Report on Design (1852) analysed the principles of design and ornament and pleaded for "more logic in the application of decoration."^[12]

Computer UX is entering its Arts and Crafts social reform era. Every internet user has [witnessed](#) the modern day Great Exhibition. History doesn't repeat, but it often rhymes: AI and Craps.

on June 08, 2026



6 comments:

Anonymous June 8, 2026 at 1:15 PM

Really enjoyed this. The idea that internet speed is itself part of the UX is something I never seen framed as the frontier.

Where I get stuck is adoption. Who actually ships the lean tier? Is the vision supposed to be enforced at protocol or browser level (a site has to expose a sub-100KB version), or does it rely on developers choosing to offer it and users choosing to use it? That second path feels like the real obstacle as most companies have no incentive to build a lightweight mode, and people on 1Gbps rarely opt down on their own. How do you see Thinnernet deployment be if you want it to be the norm?

P.S, I believe your Steve jobs alter-ego was necessary in forming this masterpiece :).

Reply Delete

Replies



Giovanni June 8, 2026 at 1:34 PM

Thanks for the kind comments. I got the idea around the time that e-ink phones were being released. Hisense and Hibreak have a dual screen format, where one is e-ink and the other one is LCD. https://www.youtube.com/watch?v=eqELxLBH_Q

Eink can use less power if it is not often refreshed, but it uses more power if it is used like an LCD. But the same idea could be applied to data. Android has a Data Saver mode and a Battery Saver mode. Technically there could be an app-lite mode which could be developed by the same company.

The only customers that would lose out might be advertisers, so manufacturers might need to charge a higher price. The Amazon Fire devices are a prime example- ad-enabled unlock screens cost less. I don't have all the solutions, but perhaps talking about it is a good starting point. :)

Delete

Reply

Anonymous June 8, 2026 at 3:33 PM

I believe the best answer is to scrap the html, css, and javascript stack, and rebuild it from the bottom. HTML was built for structured documents. The internet today is so far beyond that, and every site is using so much javascript to make their "structured document" look as far from a structured document as possible. A switch from document based internet to object based internet. Each website has many objects, each with their own state, style, events, and priority. When you click a button it only affects the state of that object, or creates/deletes an object. That would reduce the amount of data transfer by a lot. While also removing 30 years of tech debt.

Reply Delete

Replies

Anonymous June 9, 2026 at 3:26 AM

This is possible today in the cms called Exponential @
<https://github.com/se7enxweb/exponential>

Delete



Giovanni June 10, 2026 at 3:57 PM

Isn't that was AJAX does? I checked that repo- it seems like a CMS fork. Didn't really follow how it's an object based protocol..

Inferno (Plan9 lineage) is very object based ("everything is a file")

This one page was mentioned in the HN comments:
<https://github.com/mjdave/katipo>

Delete



Giovanni June 11, 2026 at 3:22 AM

I found this article which explains exponential:
<https://www.ibexa.co/blog-archive/what-is-a-digital-experience-platform-dxp-compared-to-a-content-management-system-cms>

It appears to be an alternative to CMSes by featuring more custom UX for multiple users. Overall, that's a good idea, but can also become dependent on micro targeted ads. Traditional broadcasting (one to many) retains some amount of neutrality, but I agree it's innovative in that the CMS can be customized a lot easier.

What the intro to my article was referring to, was internal ticket management systems like Jira or ZenDesk. They're more commonly known as "ITSMs", but in 2020 I didn't know there was a name for it.

Basically, the idea for my Experience Base repository was to create a two way publishing mechanism into the traditional "Knowledge Base" by learning from experiential tickets and novel issues that arise during a production mode ticket. The steps taken get recorded, either as a text long or a screen record, or both, then the KB gets footnotes/alternative solutions referencing actual fixes that work where the only relevant KB wasn't able to. The management team in charge of the KB can decide to review the changes to include them in future KB revisions, so that new employees following the same procedure become automatically aware of a more relevant fix. Obviously some fixes might not be company authorized /out of scope of the service contract, but the management doesn't have to examine each call recording, but see how the fix was made possible based on its deviations from the only available KB. Hence Experience Base.

Delete

Reply